

TASK 3 / ЗАВДАННЯ 3

MODIFYING DATA IN MYSQL: THE CARGO MANIFEST ЗМІНЮВАННЯ ДАНИХ У MYSQL: ВАНТАЖНИЙ МАНІФЕСТ

INSERT INTO, VALUES, DEFAULT, UPDATE, SET, DELETE FROM. Environment: MySQL 8.0+ (MySQL Workbench or the mysql command-line client).	INSERT INTO, VALUES, DEFAULT, UPDATE, SET, DELETE FROM. Середовище: MySQL 8.0+ (MySQL Workbench або клієнт командного рядка mysql).
--	--

Scenario · Сценарій

A supply freighter is being loaded for a run to three stations of the inner system. The manifest holds the code of the cargo unit, its name, the destination, the mass in kilograms, the priority of delivery and the loading state. The manifest is compiled item by item, corrected while the holds are being filled, and cleared of the units withdrawn before departure.	Вантажний транспорт готують до рейсу на три станції внутрішньої системи. Маніфест містить код вантажної одиниці, її назву, пункт призначення, масу в кілограмах, пріоритет доставки та стан завантаження. Маніфест укладають позицію за позицією, коригують під час заповнення трюмів і очищають від одиниць, знятих перед відльотом.
--	---

0. Environment setup · Підготовка середовища

The statements below are outside the current topics and are given ready to copy and paste. CREATE DATABASE creates the storage container, USE selects it as the default schema for the session, and CREATE TABLE defines the structure. The table is created empty: it is filled by the statements of the topics themselves. Two columns carry a DEFAULT value, which is used whenever the column is left out of a data-entry statement. The last line displays the contents of the table and may be repeated after every change to observe the effect.	Наведені нижче оператори виходять за межі поточних тем і подані у готовому до копіювання вигляді. CREATE DATABASE створює контейнер зберігання, USE обирає його як стандартну схему сеансу, а CREATE TABLE визначає структуру. Таблицю створено порожньою: її заповнюють оператори самих тем. Два стовпці мають значення DEFAULT, яке застосовується щоразу, коли стовпець не вказано в операторі введення даних. Останній рядок виводить вміст таблиці, і його можна повторювати після кожної зміни, щоб бачити результат.
---	---

```
DROP DATABASE IF EXISTS supply_run;
CREATE DATABASE supply_run CHARACTER SET utf8mb4;
USE supply_run;

CREATE TABLE cargo_item (
  item_id      INT AUTO_INCREMENT PRIMARY KEY,
  item_code    VARCHAR(10) NOT NULL,
  item_name    VARCHAR(40) NOT NULL,
  destination  VARCHAR(20) NOT NULL,
  mass_kg      DECIMAL(7,2) NOT NULL,
  priority     VARCHAR(10) NOT NULL DEFAULT 'Normal',
  status       VARCHAR(12) NOT NULL DEFAULT 'Loaded'
) ENGINE = InnoDB;

-- Contents of the manifest / Вміст маніфесту
SELECT * FROM cargo_item;
```

1. INSERT INTO

INSERT INTO adds new rows to a table. The column list written after the table name fixes the order in which the values are read, and VALUES supplies one parenthesised group per row; several groups separated by commas are added by a single statement. A column omitted from the list receives its DEFAULT value, and the key item_id is never listed, because AUTO_INCREMENT assigns it the next free number.

INSERT INTO додає до таблиці нові рядки. Перелік стовпців, записаний після назви таблиці, фіксує порядок читання значень, а VALUES подає одну взятую в дужки групу на кожен рядок; кілька груп, розділених комами, додаються одним оператором. Стовпець, пропущений у переліку, отримує своє значення DEFAULT, а ключ item_id не вказують ніколи, бо AUTO_INCREMENT надає йому наступний вільний номер.

```
-- 1.1 One row, all columns listed / Один рядок, усі стовпці вказано
INSERT INTO cargo_item (item_code, item_name, destination,
                        mass_kg, priority, status)
VALUES ('CR-001', 'Oxygen tanks', 'Luna Base', 240.00, 'High', 'Loaded');

-- 1.2 Three rows in one statement / Три рядки одним оператором
INSERT INTO cargo_item (item_code, item_name, destination,
                        mass_kg, priority)
VALUES ('CR-002', 'Water tanks', 'Mars Base', 500.00, 'High'),
       ('CR-003', 'Hull panels', 'Luna Base', 180.50, 'Normal'),
       ('CR-004', 'Soil samples', 'Orbital Lab', 45.25, 'Low');

-- 1.3 Omitted columns take defaults / Пропущені стовпці - замовчування
INSERT INTO cargo_item (item_code, item_name, destination, mass_kg)
VALUES ('CR-005', 'Medical kit', 'Mars Base', 32.75),
       ('CR-006', 'Antenna mast', 'Orbital Lab', 120.00);
```

2. UPDATE

UPDATE changes values in rows that already exist. SET carries the assignments, separated by commas, so several columns of the same row are changed in one pass. The right side of an assignment may be a constant or an expression over the current values of that row, which allows a value to be recomputed from itself. WHERE decides which rows are affected: a statement without it applies the assignments to every row of the table.

UPDATE змінює значення в рядках, які вже існують. SET несе присвоєння, розділені комами, тому кілька стовпців одного рядка змінюються за один прохід. Права частина присвоєння може бути константою або виразом над поточними значеннями цього рядка, що дозволяє перерахувати значення з нього самого. WHERE визначає, які рядки буде змінено: оператор без нього застосовує присвоєння до кожного рядка таблиці.

```
-- 2.1 A new state for one unit / Новий стан для однієї одиниці
UPDATE cargo_item
SET status = 'Sealed'
WHERE item_code = 'CR-004';

-- 2.2 Two columns for a group of rows / Два стовпці для групи рядків
UPDATE cargo_item
SET priority = 'High', status = 'Sealed'
WHERE destination = 'Mars Base';

-- 2.3 A value computed from itself / Значення, обчислене з нього самого
UPDATE cargo_item
SET mass_kg = mass_kg + 15.00
WHERE item_code = 'CR-003';
```

3. DELETE

DELETE FROM removes whole rows. The statement has no column list, because a separate value cannot be deleted: it can only be replaced through UPDATE. WHERE selects the rows to be removed, and a combined condition narrows the group to the required units. A statement without WHERE empties the table but keeps its structure, and the counter of AUTO_INCREMENT retains the value it has reached.

DELETE FROM вилучає цілі рядки. Оператор не має переліку стовпців, бо окреме значення вилучити неможливо: його можна лише замінити через UPDATE. WHERE обирає рядки для вилучення, а складена умова звужує групу до потрібних одиниць. Оператор без WHERE спорожняє таблицю, але зберігає її структуру, а лічильник AUTO_INCREMENT утримує досягнуте значення.

```
-- 3.1 One unit by its code / Одна одиниця за її кодом
DELETE FROM cargo_item
WHERE item_code = 'CR-006';

-- 3.2 A group of rows by a condition / Група рядків за умовою
DELETE FROM cargo_item
WHERE destination = 'Orbital Lab' AND mass_kg < 100;

-- 3.3 A statement without WHERE / Оператор без WHERE
DELETE FROM cargo_item;
```

Control questions · Контрольні питання

1. What value does a column receive if it is omitted from the column list of INSERT INTO?

2. Why does an UPDATE without WHERE change every row, and how is its action narrowed to one row?

3. Why does DELETE have no column list, and which statement is used to remove a single value?

1. Яке значення отримує стовпець, якщо його пропущено в переліку стовпців INSERT INTO?

2. Чому UPDATE без WHERE змінює кожен рядок і як звужити його дію до одного рядка?

3. Чому DELETE не має переліку стовпців і яким оператором вилучають окреме значення?